# Assignment No. 3 – Complex Problem-Solving Case Study

**Operating Systems | Computer Science and Engineering | 5th Sem, Section B**

**Subject Teacher:** Dr. Pallavi Wankhede | **Date of Assignment:** 24/10/2026

**Problem 1: Hospital Management and Emergency Care System**

| Group member | Name | Roll No. | Signature |
|---|---|---|---|
| 1 | | | |
| 2 | | | |

## 1. Selected Problem Statement and Real-World Context

A hospital information system handles emergency admissions, patient registrations, laboratory reports, pharmacy requests and medical imaging tasks at the same time. Incoming patient requests and diagnostic results are queued and processed by a shared server. These processes compete for CPU time, memory, and shared resources such as imaging equipment, printers and database connections.

In a hospital, wrong scheduling has real consequences: a routine registration must never delay an emergency admission, a lost lab report cannot be tolerated, and a deadlock on the imaging machine can stall patient care. The Operating System therefore has to prioritise urgent work, coordinate data flow, allocate memory efficiently and avoid unsafe resource allocation. The mini-project, a single-page *Hospital OS Dashboard* (HTML/JavaScript), simulates all four mechanisms on one shared hospital workload.

## 2. Problem Analysis, Objectives, Constraints and Assumptions

### Objectives

- Serve emergency-related processes first while keeping waiting times of other work measurable.
- Move incoming requests and results safely from producers (terminals, lab devices) to consumers (processing services) without loss or overwrite.
- Place each process in memory with low wasted space and analyse fragmentation.
- Grant shared resources only when the system remains in a safe state.

### Constraints

- Limited CPU (one core, one process at a time), limited fixed memory partitions, and a finite number of imaging machines, printers and DB connections.
- The request buffer is bounded, so producers must wait when it is full and consumers must wait when it is empty.
- Each process declares its maximum resource claim in advance (needed for Banker's Algorithm).

**Assumptions**

- Non-preemptive scheduling; the process runs to completion once dispatched.
- A lower priority number means a more urgent process (1 = emergency).
- Memory is contiguous in fixed partitions; a process releases memory and resources when it finishes.
- Resource types: A = Imaging machine, B = Printer, C = DB connection. Total = [6, 4, 5].

**Workload used for all tests**

| Process | Arrival | Burst | Priority | Memory (KB) | Max claim [A,B,C] |
|---|---|---|---|---|---|
| P0 Emergency Admission | 0 | 5 | 1 | 212 | [3,2,2] |
| P1 Patient Registration | 1 | 3 | 4 | 112 | [1,1,1] |
| P2 Lab Report | 2 | 4 | 3 | 417 | [2,1,2] |
| P3 Pharmacy Request | 3 | 2 | 3 | 90 | [1,2,1] |
| P4 Imaging Task | 4 | 6 | 2 | 426 | [3,1,2] |

Memory partitions: 100, 500, 200, 300, 600 KB (total 1700 KB).

## 3. The Four OS Concepts and Their Justification

**CPU Scheduling – Priority scheduling (non-preemptive).** Emergency admissions and imaging for critical patients must not wait behind registrations. Priority scheduling expresses this directly. Ties are broken by arrival time (FCFS). FCFS and SJF are also provided in the dashboard for comparison. Waiting time = turnaround time − burst time, and turnaround time = completion time − arrival time.

**Process Synchronization – Producer–Consumer with a bounded buffer.** Terminals, lab analysers and ward systems *produce* requests and results; processing services *consume* them. Three semaphores protect the buffer: `empty` (free slots), `full` (filled slots) and `mutex` (mutual exclusion on the buffer). This prevents buffer overflow, underflow and race conditions, so no record is lost or corrupted, which is essential for patient data.

**Deadlock Avoidance – Banker's Algorithm.** Imaging machines, printers and DB connections are non-shareable. If two processes each hold part of what the other needs, both wait forever. Banker's Algorithm uses Allocation, Maximum and Available data, computes Need = Maximum − Allocation, and grants a request only if a safe sequence exists afterwards. Avoidance is preferred over detection because in a hospital a deadlock cannot simply be recovered by killing a process.

**Memory Management – Contiguous allocation (First, Best, Worst Fit).** Each task needs a memory block. First Fit is fast, Best Fit leaves the smallest leftover, and Worst Fit leaves the largest leftover. Comparing them shows internal and external fragmentation on the same workload.

# 4. Algorithm Design, Pseudocode, Diagrams and Worked Examples

### Overall flow (diagram)

`Request arrives → Producer inserts into bounded buffer → Consumer removes (highest priority first) → Memory partition found? → Banker's safety check → Ready queue → CPU (priority) → Finish: release memory and resources`

### Priority scheduling

```
t = 0
while some process not finished:
    ready = processes with arrival <= t and not finished
    if ready is empty: t = next arrival; continue
    p = ready with smallest priority (tie: earliest arrival)
    start = t; t = t + burst(p); completion(p) = t
    TAT(p) = completion - arrival;  WT(p) = TAT - burst
```

### Producer–Consumer

```
semaphore empty = N, full = 0, mutex = 1
Producer:                    Consumer:
  wait(empty)                  wait(full)
  wait(mutex)                  wait(mutex)
  insert item                  remove item
  signal(mutex)                signal(mutex)
  signal(full)                 signal(empty)
```

### Banker's Algorithm (safety)

```
Need = Max - Allocation;  Work = Available;  Finish[i] = false
repeat:
    find i with Finish[i] = false and Need[i] <= Work
    if none: break
    Work = Work + Allocation[i];  Finish[i] = true;  add i to sequence
safe if all Finish[i] = true

Request(i, Req): if Req > Need[i] -> error
                 if Req > Available -> wait
                 pretend: Available -= Req; Alloc[i] += Req; Need[i] -= Req
                 run safety; if unsafe -> roll back and deny
```

### Memory allocation

```
for each task of size s:
    First Fit: first partition with free >= s
    Best Fit:  partition with smallest free >= s
    Worst Fit: partition with largest free >= s
    if found: free -= s  else: task waits
```

**Worked example – Banker's Algorithm.** Allocation = P0 [1,1,0], P1 [0,0,1], P2 [1,0,1], P3 [0,1,0], P4 [1,0,1]. Total allocated = [3,2,3], so Available = [6,4,5] − [3,2,3] = **[3,2,2]**.

| Process | Allocation | Max | Need = Max − Alloc |
|---|---|---|---|
| P0 | 1 1 0 | 3 2 2 | 2 1 2 |
| P1 | 0 0 1 | 1 1 1 | 1 1 0 |
| P2 | 1 0 1 | 2 1 2 | 1 1 1 |
| P3 | 0 1 0 | 1 2 1 | 1 1 1 |
| P4 | 1 0 1 | 3 1 2 | 2 1 1 |

Work = [3,2,2]. P0 (need 2 1 2 ≤ work) finishes, work = [4,3,2]. P1 finishes, work = [4,3,3]. P2 finishes, work = [5,3,4]. P3 finishes, work = [5,4,4]. P4 finishes, work = [6,4,5]. **Safe sequence: P0 → P1 → P2 → P3 → P4.**

## 5. Test Cases, Calculations and Execution Results

**Test 1 – Priority scheduling.** P0 runs 0–5. At t = 5 the ready processes are P1 (pri 4), P2 (3), P3 (3), P4 (2), so the order is P4, P2, P3, P1 (P2 before P3 because it arrived earlier).

| Process | Completion | Turnaround | Waiting |
|---|---|---|---|
| P0 Emergency Admission | 5 | 5 | 0 |
| P1 Patient Registration | 20 | 19 | 16 |
| P2 Lab Report | 15 | 13 | 9 |
| P3 Pharmacy Request | 17 | 14 | 12 |
| P4 Imaging Task | 11 | 7 | 1 |

Average waiting time = 38 / 5 = **7.6**; average turnaround time = 58 / 5 = **11.6**. For comparison, FCFS gives average waiting 5.8 and turnaround 9.8, but the emergency-related imaging task would wait 10 units instead of 1. Priority scheduling therefore trades a slightly higher average for much better service to urgent work, which is the correct choice in a hospital.

**Test 2 – Banker's request.** P2 (Lab) requests [1,0,1]. It is within its Need [1,1,1] and within Available [3,2,2]. After a trial grant, Available = [2,2,1] and the system is still safe with sequence P1 → P0 → P2 → P3 → P4, so the request is **granted**. A request larger than a process's Need is rejected, and one larger than Available must wait.

**Test 3 – Producer–Consumer.** With a buffer of 3 slots, three requests are produced (empty 3→0, full 0→3). A fourth producer call is blocked because `empty = 0`. After one consume, `empty = 1` and the blocked producer proceeds. Consuming from an empty buffer blocks the consumer on `full = 0`.

**Test 4 – Memory allocation** (tasks 212, 112, 417, 90, 426 KB):

| Algorithm | Allocated | Result |
|---|---|---|
| First Fit | 4 of 5 | Imaging (426 KB) not placed. 869 KB is free but the largest block is 300 KB: **external fragmentation** |
| Best Fit | 5 of 5 | Free space left: 10, 83, 88, 88, 174 KB (443 KB total, internal) |
| Worst Fit | 4 of 5 | Imaging not placed; large partition was split early |

Best Fit is the best for this workload.

# 6. Integration of the Algorithms into the Scenario

The dashboard's *Integrated Run* tab connects all four modules in one pipeline, using the same workload:

1. **Producer–Consumer:** each arriving request is inserted into a bounded buffer (size 3). If the buffer is full, the producer blocks.
2. **Memory management:** the consumer takes the most urgent buffered request and looks for a partition (First Fit by default). If none fits, the request waits in the buffer.
3. **Banker's Algorithm:** the admitted process first claims half its maximum resources; the remaining need is requested at dispatch. Both grants must keep the system safe.
4. **CPU scheduling:** the CPU picks the highest-priority admitted process that Banker's Algorithm allows to proceed. When it finishes, its memory and resources are released and waiting requests are retried.

**Result of the integrated run:** P0 is admitted at t = 0 (Partition 2) and runs 0–5. At t = 5 the buffer fills with P1, P2, P3 and the producer for P4 blocks, then enters once a slot frees. All four are admitted and executed in the order P4 (5–11), P2 (11–15), P3 (15–17), P1 (17–20). The results match Test 1, showing that the four mechanisms cooperate without deadlock or data loss. The emergency admission waits 0 units and no process is rejected.

The buffer controls the *rate* of requests, memory and Banker's checks control *admission*, and the scheduler controls *CPU order*. Removing any one layer would cause overflow, memory failure, deadlock or priority inversion respectively.

# 7. Conclusion

The case study shows that one hospital server needs all four OS mechanisms working together. Priority scheduling gave the emergency admission zero waiting time and the imaging task only 1 unit, at the cost of a higher average waiting time than FCFS. The bounded buffer with semaphores kept request flow lossless and race-free. Banker's Algorithm found the safe sequence P0 → P1 → P2 → P3 → P4 and would reject any unsafe grant. Best Fit placed all five tasks, while First and Worst Fit exposed fragmentation. The HTML dashboard allows every parameter to be edited and lets the examiner see each algorithm and their integration live. Possible extensions are preemptive priority scheduling with aging, multiple producers and consumers, and dynamic memory compaction.

*Source code: GitHub repository link (to be added after upload).*